

**1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquelos.**

Para este ciclo de cierre se definieron tres mini-ciclos, el primero consistió en la migración del proyecto de BlueJ a Eclipse, incluyendo la configuración del proyecto, la adición de JUnit 4 como librería y la verificación de que todo el código compilara correctamente, este mini-ciclo fue necesario porque el IDE Eclipse ofrece las herramientas de análisis estático y dinámico requeridas para la entrega.

El segundo mini-ciclo consistió en ejecutar el análisis dinámico con EclEmma, identificar y corregir los fallos en las pruebas de unidad, y verificar que la cobertura superara el 75%.

El tercer mini-ciclo consistió en ejecutar el análisis estático con PMD, identificar las violaciones de prioridad alta, corregir las que eran posibles y justificar las que no podían corregirse sin afectar la lógica del sistema.

**2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?**

Los tres mini-ciclos fueron completados satisfactoriamente, el proyecto quedó completamente migrado a Eclipse con 117 pruebas ejecutándose sin errores ni fallos, una cobertura del 89.5% y únicamente 4 violaciones de prioridad alta en PMD, todas debidamente justificadas.

**3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)**

Cortes-27 horas

**4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?**

El mayor logro de este ciclo fue alcanzar una cobertura de pruebas del 89.5%, muy por encima de la meta del 75%, mientras se corregían simultáneamente errores de lógica que habían pasado desapercibidos en ciclos anteriores, esto demuestra que las pruebas de unidad son una herramienta poderosa no solo para verificar el comportamiento del sistema sino también para detectar errores en la lógica de dominio, en particular, la corrección de la lógica de pushHierarchical y del método swap en TowerOperations mejoró significativamente la robustez del simulador.

**5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**

El mayor problema técnico fue la corrección del método swap en TowerOperations, específicamente el manejo de índices al mover las tapas junto con sus tazas después de un intercambio, el problema se manifestaba como un `IndexOutOfBoundsException` porque al remover una tapa de la lista los índices de los demás elementos cambiaban, haciendo que las posiciones calculadas previamente fueran incorrectas, se resolvió usando `indexOf` para recalcular dinámicamente la posición de cada elemento después de cada operación de

remoción, en lugar de depender de índices fijos calculados antes de la modificación de la lista.

**6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**

Estoy solo

**7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?**

La práctica más útil fue el desarrollo orientado por pruebas (TDD), contar con una suite completa de pruebas de unidad desde los ciclos anteriores permitió en este ciclo de cierre detectar inmediatamente los efectos de cada corrección sobre el resto del sistema, cada vez que se modificaba un método, las pruebas indicaban si el cambio había introducido nuevos errores en otros comportamientos. Esto fue especialmente valioso al corregir la lógica de HierarchicalCup y de swap, donde cada intento de corrección podía afectar múltiples pruebas simultáneamente.

**8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

PMD. (2024). *PMD documentation*. Recuperado de <https://pmd.github.io/pmd/index.html>

EclEmma. (2024). *EclEmma Java Code Coverage for Eclipse*. Recuperado de <https://www.eclemma.org>

Oracle. (2024). *Java SE Documentation*. Recuperado de <https://docs.oracle.com/en/java/>

La referencia más útil fue la documentación oficial de PMD (<https://pmd.github.io/pmd/index.html>), ya que permitió entender el significado de cada regla de prioridad alta reportada por la herramienta, en particular, fue útil para comprender las convenciones de nomenclatura de Java como FormalParameterNamingConvention y LocalVariableNamingConventions, y para tomar decisiones informadas sobre cuáles violaciones corregir y cuáles justificar técnicamente, como el caso de ReturnEmptyCollectionRatherThanNull donde el valor null era semánticamente necesario para la lógica del algoritmo.